

IBC Precompiled Contracts & Light Client *Strange Love*

HALBORN

IBC Precompiled Contracts & Light Client - Strange Love

Prepared by:  HALBORN

Last Updated 07/02/2024

Date of Engagement by: December 20th, 2023 - April 26th, 2024

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
11	5	0	1	3	2

TABLE OF CONTENTS

- 1. Introduction
- 2. Assessment summary
- 3. Test approach and methodology
- 4. Risk methodology
- 5. Scope
- 6. Assessment summary & findings overview
- 7. Findings & Tech Details
 - 7.1 Acknowledge sequence in ordered channel is not being increased
 - 7.2 Sendpacket handler is not checking for port ownership
 - 7.3 Reentrancy allows to replay timeout packets
 - 7.4 Gas usage in precompiled contracts is set to 1
 - 7.5 Missbehaviour handler for ibc clients is not implemented
 - 7.6 Arbitrary function call in smart contracts
 - 7.7 Redundant panics
 - 7.8 Unreachable code blocks
 - 7.9 Lack ordered_allow_timeout channels support
 - 7.10 Open todos
 - 7.11 Floating pragma
- 8. Automated Testing

1. Introduction

Strange Love engaged Halborn to conduct a security assessment on their app chain module beginning on 12/20/2023 and ending on 04/23/2024. The security assessment was scoped to the **AVAX** subnet precompiled contracts and **AVAX** light client for Cosmos provided to the Halborn team.

2. Assessment Summary

The team at Halborn was provided fifteen weeks for the engagement and assigned one full-time security engineer to assessment the security of the merge requests. The security engineers are blockchain and smart-contract security experts with advanced penetration testing, smart-contract hacking, and deep knowledge of multiple blockchain protocols.

The purpose of this assessment is to:

- Ensure that the **Golang components** operate as intended.
- Identify potential security issues with the **AVAX** subnet precompiled contracts and **AVAX** light client.

In summary, Halborn identified several relevant issues that were mostly addressed (excluding one non-addressed critical) by **Strange Love team**. In addition, Halborn also identified a lack of E2E test to verify all components are working as intended in more realistic scenarios.

3. Test Approach And Methodology

Halborn performed a combination of manual and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of the custom modules. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of structures and can quickly identify items that do not follow security best practices. The following phases and associated tools were used throughout the term of the assessment :

- Research into architecture and purpose.
- Static Analysis of security for scoped repository, and imported functions. (e.g., **staticcheck**, **gosec**, **unconvert**, **codeql**, **ineffassign** and **semgrep**)
- Static Analysis of security for scoped contracts (**slither**, **aderyn** and **4naly3er**).
- Manual Assessment for discovering security vulnerabilities on the codebase.
- Ensuring the correctness of the codebase.
- Dynamic Analysis of files and modules in scope.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (<i>s</i>)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = \min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

FILES AND REPOSITORY

(a) Repository: subnet-evm-private

(b) Assessed Commit ID: 4956e26

(c) Items in scope:

- precompile/contracts/deployerallowlist/config.go
- precompile/contracts/deployerallowlist/contract.go
- precompile/contracts/deployerallowlist/module.go
- precompile/contracts/feemanager/config.go
- precompile/contracts/feemanager/contract.go
- precompile/contracts/feemanager/module.go
- precompile/contracts/ibc/channel.go
- precompile/contracts/ibc/channel_utils.go
- precompile/contracts/ibc/client.go
- precompile/contracts/ibc/client_utils.go
- precompile/contracts/ibc/config.go
- precompile/contracts/ibc/connection.go
- precompile/contracts/ibc/connection_utils.go
- precompile/contracts/ibc/contract.go
- precompile/contracts/ibc/contract_utils.go
- precompile/contracts/ibc/mock_config.go
- precompile/contracts/ibc/module.go
- precompile/contracts/ibc/packet.go
- precompile/contracts/ibc/packet_utils.go
- precompile/contracts/ibc/port.go
- precompile/contracts/ibc/query.go
- precompile/contracts/ibc/storage.go
- precompile/contracts/ibc/verify.go
- precompile/contracts/ics20/ics20_utils.go
- precompile/contracts/nativeminter/config.go
- precompile/contracts/nativeminter/contract.go
- precompile/contracts/nativeminter/module.go
- precompile/contracts/rewardmanager/config.go
- precompile/contracts/rewardmanager/contract.go
- precompile/contracts/rewardmanager/module.go
- precompile/contracts/txallowlist/config.go
- precompile/contracts/txallowlist/contract.go
- precompile/contracts/txallowlist/module.go
- contracts/contracts/AllowList.sol
- contracts/contracts/ERC20NativeMinter.sol
- contracts/contracts/IBCApp.sol
- contracts/contracts/IBCBaseFungibleTokenApp.sol
- contracts/contracts/IBCCounterApp.sol
- contracts/contracts/ICS20Bank.sol
- contracts/contracts/ICS20BankTransferApp.sol
- contracts/contracts/interfaces/IAllowList.sol
- contracts/contracts/interfaces/IFeeManager.sol
- contracts/contracts/interfaces/IIBC.sol
- contracts/contracts/interfaces/INativeMinter.sol
- contracts/contracts/interfaces/IRewardManager.sol
- contracts/contracts/interfaces/IWarpMessenger.sol

Out-of-Scope: Test files, Example files, Third party dependencies, Economic attacks

FILES AND REPOSITORY ^

- (a) Repository: `ibc-go-strangelove-private`
- (b) Assessed Commit ID: `482c3f1`
- (c) Items in scope:
- `modules/light-clients/14-avalanche/avalanche.pb.go`
 - `modules/light-clients/14-avalanche/client_state.go`
 - `modules/light-clients/14-avalanche/codec.go`
 - `modules/light-clients/14-avalanche/consensus_state.go`
 - `modules/light-clients/14-avalanche/errors.go`
 - `modules/light-clients/14-avalanche/fraction.go`
 - `modules/light-clients/14-avalanche/genesis.go`
 - `modules/light-clients/14-avalanche/header.go`
 - `modules/light-clients/14-avalanche/keys.go`
 - `modules/light-clients/14-avalanche/misbehaviour.go`
 - `modules/light-clients/14-avalanche/misbehaviour_handle.go`
 - `modules/light-clients/14-avalanche/module.go`
 - `modules/light-clients/14-avalanche/store.go`
 - `modules/light-clients/14-avalanche/update.go`
 - `modules/light-clients/14-avalanche/verify.go`

Out-of-Scope: Test files, Example files, Third party dependencies, Economic attacks

REMEDATION COMMIT ID: ^

- `2f8ed362f8ed36`
- `2f8ed362f8ed36`
- `2f8ed362f8ed36`

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
5	0	1	3	2

SECURITY ANALYSIS	RISK LEVEL	REMEDATION DATE
ACKNOWLEDGE SEQUENCE IN ORDERED CHANNEL IS NOT BEING INCREASED	CRITICAL	SOLVED - 07/01/2024
SENDPACKET HANDLER IS NOT CHECKING FOR PORT OWNERSHIP	CRITICAL	SOLVED - 07/01/2024

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
REENTRANCY ALLOWS TO REPLAY TIMEOUT PACKETS	CRITICAL	SOLVED - 07/01/2024
GAS USAGE IN PRECOMPILED CONTRACTS IS SET TO 1	CRITICAL	SOLVED - 07/01/2024
MISSBEHAVIOUR HANDLER FOR IBC CLIENTS IS NOT IMPLEMENTED	CRITICAL	RISK ACCEPTED
ARBITRARY FUNCTION CALL IN SMART CONTRACTS	MEDIUM	RISK ACCEPTED
REDUNDANT PANICS	LOW	SOLVED - 07/01/2024
UNREACHABLE CODE BLOCKS	LOW	RISK ACCEPTED
LACK ORDERED_ALLOW_TIMEOUT CHANNELS SUPPORT	LOW	RISK ACCEPTED
OPEN TODOS	INFORMATIONAL	ACKNOWLEDGED
FLOATING PRAGMA	INFORMATIONAL	ACKNOWLEDGED

7. FINDINGS & TECH DETAILS

7.1 ACKNOWLEDGE SEQUENCE IN ORDERED CHANNEL IS NOT BEING INCREASED

// CRITICAL

Description

When an acknowledged packet is successfully processed and accepted by the `acknowledge` IBC handler, it should increment its `nextSequenceAck` counter in case of ordered channels to keep on track of every sequence of packets and accept/discard incoming packets depending on these values. However, it has been identified that `nextSequenceAck` is not being increased when a packet is accepted. Therefore, it will be impossible to handle new incoming acknowledge packets after the first one since the `setNextSequenceAck` function is setting the incoming packet's sequence as new `nextSequenceAck` in the storage, which is the same value as it was stored previously.

```
if channel.Ordering == channeltypes.ORDERED {
    nextSequenceAck, err := getNextSequenceAck(opts.accessibleState.GetStateDB(), opts.args.Packet.SourcePort)
    if err != nil {
        return fmt.Errorf("can't read next sequence ack: %w", err)
    }

    if opts.args.Packet.Sequence.Uint64() != nextSequenceAck {
        return fmt.Errorf("%w, packet sequence ≠ next ack sequence (%d ≠ %d)", channeltypes.ErrPacketSequenceMismatch, opts.args.Packet.Sequence.Uint64(), nextSequenceAck)
    }

    // All verification complete, in the case of ORDERED channels we must increment nextSequenceAck
    nextSequenceAck++

    setNextSequenceAck(opts.accessibleState.GetStateDB(), opts.args.Packet.SourcePort, nextSequenceAck)
}
```

The `setNextSequenceAck` function should use `opts.args.Packet.Sequence.Uint64() + 1` as new `NextSequenceAck`.

BVSS

AO:A/AC:L/AX:L/R:N/S:C/C:N/A:C/I:C/D:N/Y:N (1.0.0)

Recommendation

The `setNextSequenceAck` function should use `nextSequenceAck` variable as argument, since its assigned value has been increased before the new `NextSequenceAck` is set.

Remediation Plan

SOLVED: The StrangeLove team has addressed the issue by following the mentioned recommendation.

Remediation Hash

<https://github.com/ConsiderItDone/subnet-evm-private/commit/2f8ed36914a42634bbd8b58ec5495ed78d54df8f#diff-2f86252ce3ae1ebfe069f10e0dd1cccc014922697f1d13ec4d24aa984bc93a9>

7.2 SENDPACKET HANDLER IS NOT CHECKING FOR PORT OWNERSHIP

// CRITICAL

Description

When `sendPacket` IBC handler is executed to send a new packet to a different chain, there are some verification and validations of the packets that need to be performed in order to not allow sending malformed packets, using incorrect channels, ports or connections. One of the components involved in these verifications is the source port, since ports allow to bind modules/applications to handle different IBC packets, they are also sources of messages/packets transmitted through channels, which basically connects two ports on different chains. For this reason, it's crucial to validate in `sendPacket` IBC handler that the owner of the source port, which is attached to an application/module (in this case, smart contracts), specified in the packet's structure is the caller of `sendPacket`. Otherwise, arbitrary calls could be made impersonating modules and applications by setting their ports in IBC packets. This is exactly what happens in `_sendPacket` function, which is in charge of validating all these conditions to guarantee a packet can be sent safely through a channel.

```
ok, err := GetCapability(opts.accessibleState.GetStateDB(), opts.args.SourcePort, opts.args.SourceChannel)
if err != nil {
    return err
}
if !ok {
    return fmt.Errorf("caller does not own port capability for port ID %s, %w", opts.args.SourcePort, err)
}
```

The `GetCapability` just returns `true/false` in case a channel's capability is equal to `[]byte{1}`, which is set by default when a channel is opened successfully.

```
func GetCapability(db contract.StateDB, portID, channelID string) (bool, error) {
    state, err := GetState(db, ChannelCapabilitySlot(portID, channelID))
    if err != nil {
        return false, err
    }
    if !bytes.Equal(state, []byte{1}) {
        return false, ErrBadCapability
    }
    return true, nil
}
```

This function does not verify whether the caller of the `sendPacket` function is the same as the smart contract bound to the source port specified in the packet. Therefore, any user could be allowed to send packets on behalf of arbitrary ports in the IBC protocol.

BVSS

[AO:A/AC:L/AX:L/R:N/S:C/C:C/A:N/I:C/D:N/Y:N \(10.0\)](#)

Recommendation

It is recommended to add a check that verifies that the function's caller is the same as the port's owner. If this condition is not met, revert the transaction.

Remediation Plan

SOLVED: The **StrangeLove** team has addressed the issue by following the mentioned recommendation.

Remediation Hash

<https://github.com/ConsiderItDone/subnet-evm-private/commit/2f8ed36914a42634bbd8b58ec5495ed78d54df8f#diff-2f86252ce3ae1ebfe069f10e0dd1cccc014922697f1d13ec4d24aa984bc93a9>

7.3 REENTRANCY ALLOWS TO REPLAY TIMEOUT PACKETS

// CRITICAL

Description

The `sendPacket` IBC handler stores a commitment in the source chain every time that a packet is sent, this commitment is deleted when its associated packet has been acknowledged or timed-out on its respective handlers. This commitment helps to prevent from replay attacks, since `acknowledge` and `timeout` IBC handlers will verify whether this commitment exists or not for a specific packet to check if the chain has already processed it. However, it exists an execution flow that would allow replaying timeout packets by making reentrant calls.

```
_, remainingGas, err = accessibleState.CallFromPrecompile(ContractAddress, recvAddr, data, remainingGas, b
if err != nil {
    return nil, remainingGas, err
}
// Delete packet commitment
if err = TimeoutExecuted(accessibleState, inputStruct.Packet); err != nil {
    return nil, remainingGas, err
}
```

When the `timeout` function is executed in the precompiled contract, there are many validations and sanity checks run over the packet to verify if it is legit. After all these checks are passed, the precompiled contract makes a call to the smart contract in charge to handle the accepted timeout packet, thus delegates the execution to an arbitrary smart contract bound to a specific port. Once the smart contract's execution has ended without reverting, the packet commitment is deleted from the storage by executing the `TimeoutExecuted` function.

This execution's pattern opens a frame where any smart contract executed during `CallFromPrecompile` context could call recursively to `timeout` function, and it will process the timeout packet again since its associated commitment hasn't been deleted.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:H/Y:H (10.0)

Recommendation

It is recommended to use the **Checks-Effects-Interactions Pattern** (CEI) in order to prevent from this kind of issues, deleting the commitment before the execution of `CallFromPrecompile` function.

Remediation Plan

SOLVED: The StrangeLove team has addressed the issue by following the mentioned recommendation.

Remediation Hash

<https://github.com/ConsiderItDone/subnet-evm-private/commit/2f8ed36914a42634bbd8b58ec5495ed78d54df8f#diff-31215179c08917d2be5f4c3044b57c85b204133f21566c7938bc8d291d172dff>

7.4 GAS USAGE IN PRECOMPILED CONTRACTS IS SET TO 1

// CRITICAL

Description

It's well-known the purpose of gas in EVM based blockchain, allowing to a decentralized system to measure and limit the amount of computation required for a task, as it is executing smart contract. In terms of security, this feature allows preventing from spamming attacks which would overload the blockchain or even cause a Denial-Of-Service (DOS) with catastrophic consequences.

AVAX allows developers to work with Subnets that implement custom precompiled contracts, these special contracts perform different customized tasks in a more optimized way, out of the regular things that EVM allows to do. These precompiled contracts also require from specifying an amount of gas to consume that should vary depending on the computation and the amount of memory as used in their execution. Otherwise, an attacker would be able to spam computationally expensive functions without any drawbacks in its actions, this could lead to a slowdown of the entire chain or even halt.

In the case of the scoped precompiled contracts, there are a few new custom functions implemented to be interacted through EVM which consume the following amount of gas per call:

```
const (
    CreateClientGasCost  uint64 = 1
    UpdateClientGasCost  uint64 = 1
    UpgradeClientGasCost uint64 = 1

    ConnOpenAckGasCost    uint64 = 1
    ConnOpenConfirmGasCost uint64 = 1
    ConnOpenInitGasCost   uint64 = 1
    ConnOpenTryGasCost    uint64 = 1

    ChanOpenInitGasCost    uint64 = 1
    ChanOpenTryGasCost     uint64 = 1
    ChannelCloseConfirmGasCost uint64 = 1
    ChannelCloseInitGasCost  uint64 = 1
    ChannelOpenAckGasCost   uint64 = 1
    ChannelOpenConfirmGasCost uint64 = 1

    BindPortGasCost uint64 = 1

    RecvPacketGasCost    uint64 = 1
    SendPacketGasCost    uint64 = 1
    AcknowledgementGasCost uint64 = 1
    TimeoutGasCost        uint64 = 1
    TimeoutOnCloseGasCost uint64 = 1

    QueryClientStateGasCost    uint64 = 1
    QueryConsensusStateGasCost uint64 = 1
    QueryConnectionGasCost     uint64 = 1
    QueryChannelGasCost        uint64 = 1

    QueryPacketCommitmentGasCost    uint64 = 1
    QueryPacketAcknowledgementGasCost uint64 = 1
)
```

As it can be seen, all these functions consume 1 unit of gas per call. This without having in consideration the amount of computation required per call nor the read/writes of memory used during their executions.

BVSS

AO:A/AC:M/AX:L/R:N/S:U/C:H/A:C/I:H/D:N/Y:N (9.2)

Recommendation

It is recommended to set a higher amount of gas to request per function execution, taking into consideration the amount of computation required to be executed successfully, and add dynamic fees depending on the amount of memory used to during the function's execution. For example, in some precompiled contracts in EVM there are some functions which apply dynamic gas fees depending on the argument's length.

Remediation Plan

SOLVED: The StrangeLove team has addressed the issue by setting different gas values to precompiled function calls.

Remediation Hash

<https://github.com/ConsiderItDone/subnet-evm-private/commit/2f8ed36914a42634bbd8b58ec5495ed78d54df8f>

7.5 MISBEHAVIOUR HANDLER FOR IBC CLIENTS IS NOT IMPLEMENTED

// CRITICAL

Description

An IBC client requires to be constantly sending **update** messages to track the status of the client and the remote state machine. This entails accepting only the state updates that were agreed upon by the remote state machine's consensus algorithm. In the scoped AVAX Subnet, the precompiled contract's function in charge of this is **_updateClient**. This function allows to every relayer to update the status of the remote state machine in any moment.

However, there is other case where the aforementioned status should be updated, when a **Misbehaviour** message is received. Following the IBC specification:

The Tendermint light client defines Misbehaviour as two different types of situations: a situation where two conflicting Headers with the same height have been submitted to update a client's ConsensusState within the same trusting period, or that the two conflicting Headers have been submitted at different heights but the consensus states are not in the correct monotonic time ordering (BFT time violation). More explicitly, updating to a new height must have a timestamp greater than the previous consensus state, or, if inserting a consensus at a past height, then time must be less than those heights which come after and greater than heights which come before.

Therefore, the receiver of this message should handle this message by checking if it's a valid message and freeze the client so that calling the **Status** function on the associated client state no longer returns **Active**.

```
switch msg := clientMsg.(type) {
    case *ibctm.Header:
        clientState.LatestHeight = msg.GetHeight().(clienttypes.Height)
        consensusState.Timestamp = msg.GetTime()
        consensusState.Root = commitmenttypes.NewMerkleRoot(msg.Header.GetAppHash())
        consensusState.NextValidatorsHash = msg.Header.NextValidatorsHash
    case *ibctm.Misbehaviour:
        // ToDo: implement me
        return fmt.Errorf("not implemented")
    default:
        return clienttypes.ErrUpdateClientFailed
}
```

As it can be observed, the switch statement in charge to route messages to their respective handlers does not implement a proper handler to **Misbehaviour** messages. For this reason, the chain will never be able to handle **Misbehaviour** messages.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:H/I:H/D:N/Y:N (9.4)

Recommendation

It is necessary to implement a handler for **Misbehavior** messages as it is described in the IBC specification to handle possible errors during IBC communication when a client is being updated.

Remediation Plan

RISK ACCEPTED: The StrangeLove team has accepted the risk of this issue.

7.6 ARBITRARY FUNCTION CALL IN SMART CONTRACTS

// MEDIUM

Description

The `ICS20BankTransferApp` smart contract allows to bind itself to an IBC port through the `bindPort` function implemented on it. However, since there is no access control to these functions, and it allows specifying an arbitrary address to execute `bindPort` function, any user in the chain can trigger this function to call arbitrary contracts leading to undesired actions over this contract.

```
function bindPort(address someAddr, string memory portId) external {
    PortBinder(someAddr).bindPort(portId);
}
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:M/A:N/I:L/D:L/Y:L (6.9)

Recommendation

It is recommended to implement an access control to the mentioned function where, for example, only an owner could execute it, or replace the argument which allows setting an arbitrary address with a static address to prevent from calling arbitrary smart contracts.

Remediation Plan

RISK ACCEPTED: The `StrangeLove` team has accepted the risk of this issue.

7.7 REDUNDANT PANICS

// LOW

Description

It has been identified a condition where the Go program execution panics when it's possible to return in a clean way with errors. The `_timeoutOnClose` function, part of the IBC precompiled contracts, verifies the type of channel specified in the received message, if the specified channel is not **ORDERED** or **UNORDERED**, it panics the process instead of returning an error, preventing from handling the error message correctly. This also happens in `TimeoutPacket` function.

```
switch channel.Ordering {
case channeltypes.ORDERED:
    // check that packet has not been received
    if nextSequenceRecv > packet.GetSequence() {
        return fmt.Errorf("%w, packet already received, next sequence receive > packet sequence (%d > %d)",
            channeltypes.ErrInvalidChannelOrdering, channel.Ordering.String(), nextSequenceRecv, packet.GetSequence())
    }

    // check that the recv sequence is as claimed
    err = VerifyNextSequenceRecv(
        marshaler, connectionEnd, proofHeight, proof,
        packet.GetDestPort(), packet.GetDestChannel(), nextSequenceRecv, accessibleState,
    )
case channeltypes.UNORDERED:
    err = VerifyPacketReceiptAbsence(
        marshaler, connectionEnd, proofHeight, proof,
        packet.GetDestPort(), packet.GetDestChannel(), packet.GetSequence(), accessibleState,
    )
default:
    panic(fmt.Errorf("%w, %s", channeltypes.ErrInvalidChannelOrdering, channel.Ordering.String()))
}
```

```
switch channel.Ordering {
case channeltypes.ORDERED:
    // check that packet has not been received
    if opts.args.NextSequenceRecv.Uint64() > packet.GetSequence() {
        return fmt.Errorf("%w, packet already received, next sequence receive > packet sequence (%d > %d)",
            channeltypes.ErrInvalidChannelOrdering, channel.Ordering.String(), opts.args.NextSequenceRecv.Uint64(), packet.GetSequence())
    }

    // check that the recv sequence is as claimed
    err = VerifyNextSequenceRecv(
        marshaler, connectionEnd, proofHeight, opts.args.ProofUnreceived,
        packet.GetDestPort(), packet.GetDestChannel(), opts.args.NextSequenceRecv.Uint64(), opts.accessibleState,
    )
case channeltypes.UNORDERED:
    err = VerifyPacketReceiptAbsence(
        marshaler, connectionEnd, proofHeight, opts.args.ProofUnreceived,
        packet.GetDestPort(), packet.GetDestChannel(), packet.GetSequence(), opts.accessibleState,
    )
default:
    panic(fmt.Errorf("%w, %s", channeltypes.ErrInvalidChannelOrdering, channel.Ordering.String()))
}
```

Recommendation

It is recommended to use `return` instead of `panic` to handle the error correctly.

Remediation Plan

SOLVED: The StrangeLove team has addressed the issue by setting different gas values to precompiled function calls.

Remediation Hash

<https://github.com/ConsiderItDone/subnet-evm-private/commit/2f8ed36914a42634bbd8b58ec5495ed78d54df8f#diff-31215179c08917d2be5f4c3044b57c85b204133f21566c7938bc8d291d172dff>

7.8 UNREACHABLE CODE BLOCKS

// LOW

Description

It has been identified some code blocks that are never executed due to impossible conditions. For example, in `_sendPacker` function handler there is a code block which will never be executed do to its conditions associated.

```
ok, err := GetCapability(opts.accessibleState.GetStateDB(), opts.args.SourcePort, opts.args.SourceChannel)
if err != nil {
    return err
}
if !ok {
    return fmt.Errorf("caller does not own port capability for port ID %s, %w", opts.args.SourcePort, err)
}
```

The `GetCapability` function will never return `false` without returning an error, otherwise the `err != nil` condition will be met.

```
func GetCapability(db contract.StateDB, portID, channelID string) (bool, error) {
    state, err := GetState(db, ChannelCapabilitySlot(portID, channelID))
    if err != nil {
        return false, err
    }
    if !bytes.Equal(state, []byte{1}) {
        return false, ErrBadCapability
    }
    return true, nil
}
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:L/D:N/Y:N (3.1)

Recommendation

It is recommended to remove this kind of code blocks to reduce the code's complexity.

Remediation Plan

RISK ACCEPTED: The StrangeLove team has accepted the risk of this issue.

7.9 LACK ORDERED_ALLOW_TIMEOUT CHANNELS SUPPORT

// LOW

Description

Following the ICS-004 specification regarding channel and packet semantics, there are three type of packet supported by IBC:

- Ordered, which is supported by the IBC precompiled contracts.
- Unordered, which is supported by the IBC precompiled contracts.
- Ordered_allow_timeout, which is NOT supported by the IBC precompiled contracts.

The aforementioned specification states: *an ordered_allow_timeout channel is a less strict version of the ordered channel. Here, the channel logic will take a best effort approach to delivering the packets in order. In a stream of packets, the channel will relay all packets in order and if a packet in the stream times out, the timeout logic for that packet will execute and the rest of the later packets will continue processing in order. Thus, we do not close the channel on a timeout with this channel type.*

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

It is recommended to allow the usage of all types of channels to achieve a better compatibility with the whole IBC ecosystem.

Remediation Plan

RISK ACCEPTED: The StrangeLove team has accepted the risk of this issue.

7.10 OPEN TODOS

// INFORMATIONAL

Description

It has been identified Open TO-DOs in the scoped files. Open TO-DOs can point to architecture or programming issues that still need to be resolved. Often these kinds of comments indicate areas of complexity or confusion for developers. This provides value and insight to an attacker who aims to cause damage to the protocol.

Score

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Consider resolving the TO-DOs before deploying code to a production context. Use an independent issue tracker or other project management software to track development tasks.

Remediation Plan

ACKNOWLEDGED: The StrangeLove team has acknowledged this issue.

7.11 FLOATING PRAGMA

// INFORMATIONAL

Description

Smart contracts in scope use the floating pragma ^0.8.0. Contracts should be deployed with the same compiler version and flags that they have been tested with thoroughly. Locking the pragma helps to ensure that contracts do not accidentally get deployed using, for example, either an outdated compiler version that might introduce bugs that affect the contract system negatively or a pragma version too new which has not been extensively tested.

Score

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Consider locking the pragma version with known bugs for the compiler version by removing the caret (^) symbol. When possible, do not use floating pragma in the final live deployment. Specifying a fixed compiler version ensures that the bytecode produced does not vary between builds. This is especially important if you rely on bytecode-level verification of the code.

Remediation Plan

ACKNOWLEDGED: The StrangeLove team has acknowledged this issue.

8. AUTOMATED TESTING

Halborn used automated testing techniques to enhance coverage of certain areas of the scoped component. Among the tools used were **slither** (for smart contracts in scope), **staticcheck**, **gosec**, **semgrep**, **codeQL** and **Nancy**. After Halborn verified all the contracts and scoped structures in the repository and was able to compile them correctly, these tools were leveraged on scoped structures. With these tools, Halborn can statically verify security related issues across the entire codebase.

```
subnet-evm-private/precompile/contracts/ibc/channel_utils.go
(207, 6) U1000 func setConsensusMetadata is unused

subnet-evm-private/precompile/contracts/ibc/client_test.go
(144, 3) S1021 should merge variable declaration with assignment on next line

subnet-evm-private/precompile/contracts/ibc/client_utils.go
(239, 6) U1000 func checkForMisbehaviour is unused
(261, 4) S1008 should use 'return !reflect.DeepEqual(existingConsState, tmHeader.ConsensusState())' instead of 'if reflect.DeepEqual(existingConsState, tmHeader.ConsensusState()) { return false }; return true'

/subnet-evm-private/precompile/contracts/ibc/contract_utils.go
(126, 2) U1000 field consState is unused
(127, 2) U1000 field previousHeight is unused
(163, 16) ST1005 error strings should not be capitalized
(166, 14) ST1005 error strings should not be capitalized
```

The **Staticcheck** output does not show any relevant issue.

pkg:golang/google.golang.org/protobuf@v1.31.0	
1 known vulnerabilities affecting installed version	
[CVE-2024-24786] CWE-835: Loop with Unreachable Exit Condition ('Infinite Loop')	
Description	The protojson.Unmarshal function can enter an infinite loop when unmarshaling certain forms of invalid JSON. This condition can occur when unmarshaling into a message which contains a google.protobuf.Any value, or when the UnmarshalOptions.DiscardUnknown option is set.
OSS Index ID	CVE-2024-24786
CVSS Score	7.5/10 (High)
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

[CVE-2022-23328] CWE-400: Uncontrolled Resource Consumption ('Resource Exhaustion')	
Description	A design flaw in all versions of Go-Ethereum allows an attacker node to send 5120 pending transactions of a high gas price from one account that all fully spend the full balance of the account to a victim Geth node, which can purge all of pending transactions in a victim node's memory pool and then occupy the memory pool to prevent new transactions from entering the pool, resulting in a denial of service (DoS).
OSS Index ID	CVE-2022-23328
CVSS Score	7.5/10 (High)
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

[CVE-2023-40591] CWE-400: Uncontrolled Resource Consumption ('Resource Exhaustion')	
Description	go-ethereum (geth) is a golang execution layer implementation of the Ethereum protocol. A vulnerable node, can be made to consume unbounded amounts of memory when handling specially crafted p2p messages sent from an attacker node. The fix is included in geth version `1.12.1-stable`, i.e., `1.12.2-unstable` and onwards. Users are advised to upgrade. There are no known workarounds for this vulnerability. Sonatype's research suggests that this CVE's details differ from those defined at NVD. See https://ossindex.sonatype.org/vulnerability/CVE-2023-40591 for details
OSS Index ID	CVE-2023-40591
CVSS Score	7.5/10 (High)
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

[CVE-2023-42319] CWE-noinfo

Description	Geth (aka go-ethereum) through 1.13.4, when --http --graphql is used, allows remote attackers to cause a denial of service (memory consumption and daemon hang) via a crafted GraphQL query. NOTE: the vendor's position is that the "graphql endpoint [is not] designed to withstand attacks by hostile clients, nor handle huge amounts of clients/traffic.
-------------	---

OSS Index ID	CVE-2023-42319
--------------	----------------

CVSS Score	7.5/10 (High)
------------	---------------

CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
-------------	--

pkg:golang/github.com/consensys/gnark-crypto@v0.10.0
1 known vulnerabilities affecting installed version

[CVE-2023-44273] CWE-502: Deserialization of Untrusted Data

Description	Consensys gnark-crypto through 0.11.2 allows Signature Malleability. This occurs because deserialisation of EdDSA and ECDSA signatures does not ensure that the data is in a certain interval.
-------------	--

OSS Index ID	CVE-2023-44273
--------------	----------------

CVSS Score	9.8/10 (Critical)
------------	-------------------

CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
-------------	--

pkg:golang/github.com/dvsekhvalnov/jose2go@v1.5.0
1 known vulnerabilities affecting installed version

[CVE-2023-50658] CWE-834: Excessive Iteration	
Description	The jose2go component before 1.6.0 for Go allows attackers to cause a denial of service (CPU consumption) via a large p2c (aka PBES2 Count) value.
OSS Index ID	CVE-2023-50658
CVSS Score	7.5/10 (High)
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

pkg:golang/github.com/hashicorp/go-getter@v1.7.1
1 known vulnerabilities affecting installed version

[CVE-2024-3817] CWE-88: Argument Injection or Modification	
Description	HashiCorp's go-getter library is vulnerable to argument injection when executing Git to discover remote branches. This vulnerability does not affect the go-getter/v2 branch and package. Sonatype's research suggests that this CVE's details differ from those defined at NVD. See https://ossindex.sonatype.org/vulnerability/CVE-2024-3817 for details
OSS Index ID	CVE-2024-3817
CVSS Score	9.8/10 (Critical)
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

The **Nancy** output shows several dependencies that contain identified vulnerabilities an should be considered.
The rest of tools didn't show any relevant issue.

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.

